TRABAJO PREVIO

Ejercicio previo 1: abriendo ficheros concurrentemente ("ejprev1.cc")

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <cstring>
#include <vector>
#include <pthread.h>

using namespace std;

void *hilo(void *arg)
{
   const char *fichero = (const char *) arg;
   stringstream sout;
   ifstream ifs{fichero};

   if (not ifs)
   {
       sout << "Fichero '" << fichero << "': no se ha podido abrir para leer ("
            << strerror(errno) << ")\n";
       cout << sout.str();

       return (void *) 1;
   }
   else
   {
       sout << "Fichero '" << fichero << "': éxito\n";
       cout << sout.str();

       return (void *) 0;
   }
}

int main(int argc, char *argv[])
{
   vector<pthread_t> id;

   for (int i = 1; i < argc; ++i)
   {
       pthread_t t;
       pthread_create(&t, NULL, hilo, argv[i]);
       id.push_back(t);
   }

   int errores_totales = 0;
```

```
    for (auto & t : id)
    {
        void *r;
        pthread_join(t, &r);

        long resultado = (long) r;
        errores_totales += resultado;
    }

    return errores_totales;
}
```

Ejercicio previo 2: contando líneas con "getline" ("ejprev2.cc")

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <cstring>
#include <vector>
#include <pthread.h>

using namespace std;

void *hilo(void *arg)
{
    const char *fichero = (const char *) arg;
    stringstream sout;

    ifstream ifs{fichero};

    if (not ifs)
    {
        sout << "Fichero '" << fichero << "': no se ha podido abrir para leer ("
             << strerror(errno) << ")\n";
        cout << sout.str();
        return (void *) 1;
    }
    else
    {
        string linea;
        int contador_lineas = 0;

        while (getline(ifs, linea))
        {
            contador_lineas++;
        }
```

```
        sout << "Fichero '" << fichero << "': tiene " << contador_lineas << "
líneas\n";
        cout << sout.str();

        return (void *) 0;
    }
}

int main(int argc, char *argv[])
{
    vector<pthread_t> id;

    for (int i = 1; i < argc; ++i)
    {
        pthread_t t;
        pthread_create(&t, NULL, hilo, argv[i]);
        id.push_back(t);
    }

    int errores_totales = 0;

    for (auto & t : id)
    {
        void *r;
        pthread_join(t, &r);

        long resultado = (long) r;
        errores_totales += resultado;
    }

    return errores_totales;
}
```